

Partial Evaluation + Global Analyzer = Separate Analyzer

Gyuhyeok Oh
Seoul National University

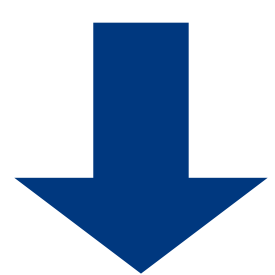
“How to systematically derive separate analyzer?”

Key Concept

Partial evaluation of monolithic analyzer yields separate abstract compiler.

Monolithic Analysis

- Monolithic Analysis guarantees global precision but lacks the flexibility for reuse and scaling
- $\mathcal{E}^\#$ takes a context and analyzes one module.
 $\mathcal{E}^\# : \text{Module} \times \text{Env}^\# \rightarrow \text{Env}^\#$
- The analysis of M_1 is **necessary** for analysis of M_2



Systematically derive with PE
 $\llbracket \mathcal{PE}(e, i_1) \rrbracket i_2 = \llbracket e \rrbracket \langle i_1, i_2 \rangle$

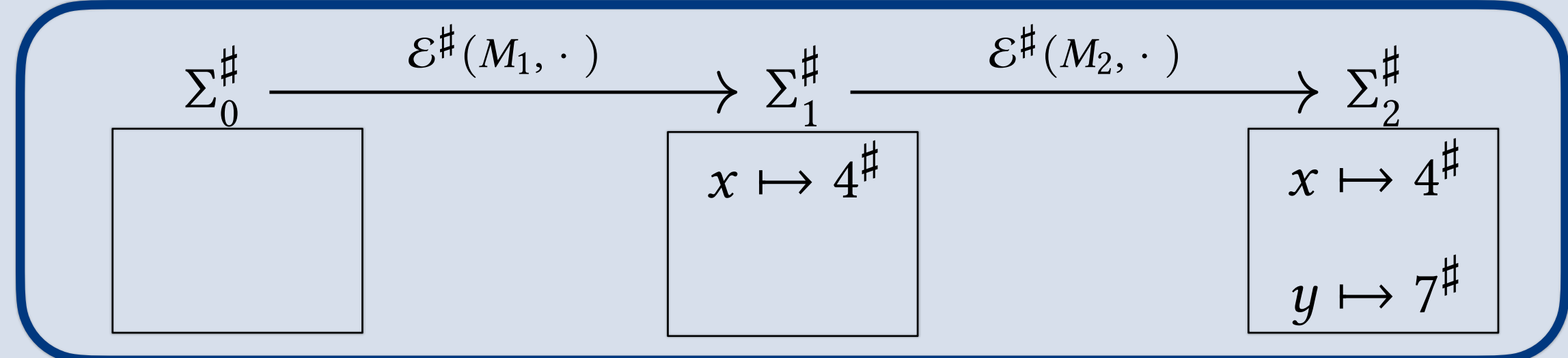
Separate Analysis

- Separate Analysis promotes reusability and modularity **without losing precision.**
- Analyzing M_2 can be done independently to the analysis of M_1
- **Residual code** $\mathcal{A}_M \triangleq \mathcal{PE}(\mathcal{E}^\#, M)$
is separate analysis result.
- Whole program analysis achieved by linking residual codes (separate analyses) in dependency order.

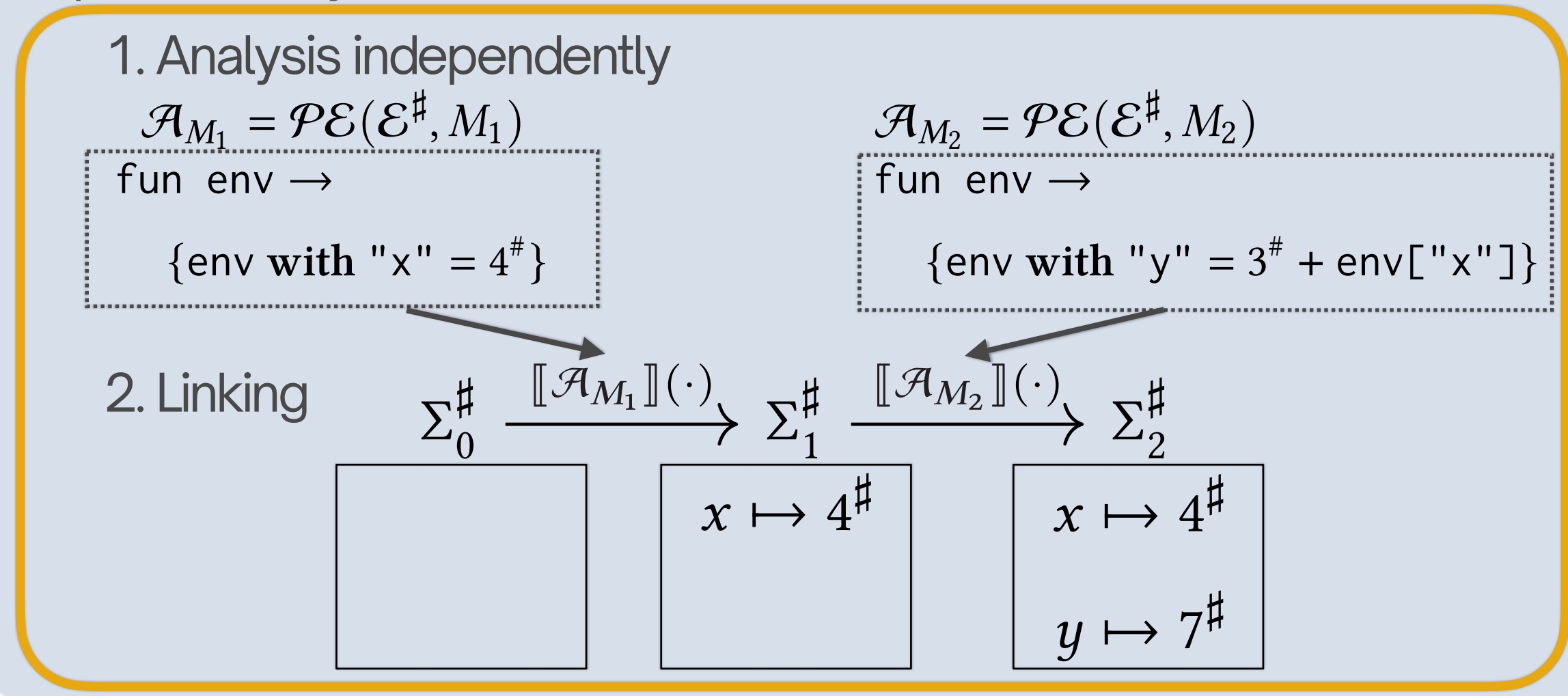
Monolithic vs. Separate: Analysis on

$M_1 : x = 4$
 $M_2 : y = 1 + 2 + x$

Monolithic Analysis



Separate Analysis



Implementation

Implementation with manual partial evaluation via meta-programming

- Domain knowledge (e.g., lattice properties) enables optimizations unattainable by standard PE.
- Developing a full-scale partial evaluator is complex and costly.

```
module M1 =
{ let x = 20;
  let f = fun x -> x + 1 }

module M2 =
{ let res1 = M1.f M1.x;
  let res2 = M1.f (4 + 1) }
```

Separate analysis

Separate analysis result of res1

```
fun link_env ->
(* Dynamic Lookup for both function and argument *)
let abs_func = LinkEnv.lookup link_env "M1.f" in
let abs_arg = LinkEnv.lookup link_env "M1.x" in

(* Apply and Join *)
abs_func |> map_join (fun closure ->
  apply closure abs_arg )
```

Separate analysis result of res2

```
fun link_env ->
(* Dynamic Lookup only for function *)
let abs_func = LinkEnv.lookup link_env "M1.f" in
let const_arg = [5, 5] in < Pre-computed Constant

(* Apply and Join *)
abs_func |> map_join (fun closure ->
  apply closure const_arg)
```

- ✓ The final result, obtained via linking residual code, is equivalent to the monolithic analysis result.
- ✓ Reconstructs the **global fixpoint** across module boundaries **without loss of precision.**

Future work

- Complex Module Semantics
 - Cyclic Dependencies: Recursive modules
 - Implicit Interactions: Enabling communication through shared state
- Optimization of residual analyzer
- Building benchmarks for empirical evaluation